Republic of the Philippines
CEBU TECHNOLOGICAL UNIVERSITY
ARGAO CAMPUS
Ed Kintanar Street, Lamacan, Argao, Cebu
Website:http://www.argao.ctu.edu.ph E-mail:cdargao@ctu.edu.ph
Phone No.: (032) 401-0737 local 1700
BAGONG PILIPINAS

Cross Platform Script Development
Week 3
Information Technology Department

Name:	
Course:	**BSIT**
Year & Section:	
Date:	
Topic:	JavaScript Designs Patterns – Factory Design
Activity:	2

Notes:

What are design patterns?
- Reusable templates known as design patterns assist programmers in resolving certain software design issues.

Importance of Design Patterns
- **Code Reusability** – to promote the reuse of successful solutions, reducing the need to reinvent the wheel and speeding up development.

- **Scalability** – provides a foundation for scalable and extensible code, allowing applications to grow and adapt to changing requirements.

- **Maintainability** – code becomes more modular and easier to understand, debug, and maintain over time.

3 Kinds of JavaScript Design Patterns
Factory Pattern - a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created. In simpler terms, it's a way to create objects without explicitly specifying their concrete classes.

Singleton Pattern – it ensures that a class has only one instance and provides a global point of access to that instance.

Strategy Pattern – a kind of behavioral design pattern that allows defining a family of algorithms, encapsulating each one, and making them interchangeable.

Analogy: Think of it as a coffee shop, you don't have to think or know what kind of coffee you can order

Why Use Factory Pattern?

- **Centralized Object Creation** → Creation logic is in one place.
- **Avoids Repetition** → No need to repeat new Car("sedan") everywhere.
- **Extensibility** → Easy to add new object types later.
- **Decoupling** → Clients don't need to know *how* objects are created.

Factory Pattern Code

```
//Note
class CarFactory {
  createCar(type) {
    switch (type) {
      case "sedan": return { type: "Sedan", wheels: 4, fuel: "Gasoline" };
      case "suv": return { type: "SUV", wheels: 4, fuel: "Diesel" };
      default: return { type: "Generic Car", wheels: 4, fuel: "Unknown" };
    }
  }
}

function main(){
    const factory = new CarFactory();
    console.log(factory.createCar("sedan"));

}

main();
```

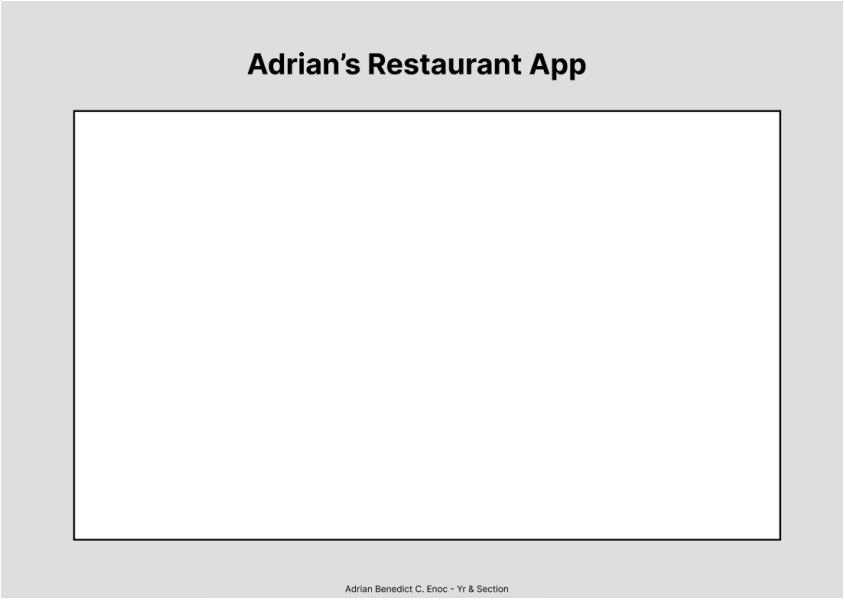
Activity: Restaurant Ordering System using Factory Pattern

Create a JavaScript mini-app that allows the user to order and display final order summary of the items selected in a HTML Page.

Requirements:

1. On title of the app it would show "YourFirstName Restaurant App"
2. On the footer or bottom of the app, it would display your full name inside an p html tag.
3. Create a restaurant factory that can handle 3 categories:
    - foodFactory (Spaghetti, Fried Chicken, Burrito, etc.)
    - drinkFactory (Water, Sprite, Ice Tea, etc.)
    - dessertFactory (Ice Cream, Cake, Donut, etc.)
4. Each item must have:
    - name
    - price
    - ingredients
5. Allows the user to select via the html page from all of the 3 categories.
6. Generate a final order summary with total price

Sample Output:



Submission of Activity

1. Upload to Google Classroom: compress your work on .zip or .7zip format only example: activity-2.zip
2. If you have questions regarding on this activity, just send a message at the google classroom.